Catch-All Query Anti-Pattern

#sql-server #performance #parameter-sniffing #refactoring

What the Pattern Looks Like

A stored procedure with several optional parameters, where the WHERE clause uses `OR @Param IS NULL` to mean "treat this parameter as a wildcard if it was not supplied":

```
Create Procedure GetSortationItems
   @Barcode varchar(100) = Null,
   @SortationCode varchar(30) = Null
As
Begin
   Select Barcode, SortationCode
   From SrtSortationItem With (NoLock)
   Where (@Barcode Is Null Or Barcode = @Barcode)
     And (@SortationCode Is Null Or SortationCode = @SortationCode)
End
```

The intent is reasonable: one proc that handles "look up by barcode," "look up by sortation code," "look up by both," and "return everything." The cost is steep. The optimizer compiles a single plan for the proc, and that plan has to handle every parameter combination, including the "all NULLs return everything" case. To stay correct in the worst case, the optimizer chooses a table or index scan. Every call pays the scan cost, even the calls that supply both parameters and want a single-row seek.

The pattern is sometimes called the kitchen-sink query, the catch-all query, or the parameterized search query.

Why It Hurts

The optimizer compiles plans at compile time, not at runtime. When it sees `(@Param Is Null Or Col = @Param)`, it cannot evaluate `@Param Is Null` until the query runs. Compile time has to plan for both branches of the OR. Both branches have to be in the plan, which means the predicate cannot be pushed into an index seek that requires `Col = constant`. The result is a full scan of the table or its widest covering index, every call.

Three concrete consequences:

- **Logical reads scale with table size, not result size.** A barcode lookup that should return 1 row reads thousands of pages because the engine has to evaluate every row.
- **The plan caches once and stays bad.** Every call uses the same scan plan, so there is no parameter sniffing volatility, just consistently bad performance.
- **Adding indexes does not help.** The optimizer has the indexes available but cannot use them efficiently because the `Or @Param Is Null` clause prevents seek-based access.

In production, you see this as a proc with low plan variability, high logical reads per execution, high CPU, and a plan that shows a Clustered Index Scan or Index Scan where common sense says it should be an Index Seek.

The Three Canonical Fixes

Each fix has a specific place where it shines. The right choice depends on the number of optional parameters and how often the proc is called.

Fix 1: IF/ELSE Branching

Best when the number of parameters is small (two or three) and the number of combinations is manageable (four to eight). Each combination becomes its own SELECT, each with a clean cached plan.

```
If @Barcode Is Not Null And @SortationCode Is Not Null
Begin
   Select Barcode, SortationCode
   From SrtSortationItem With (NoLock)
   Where Barcode = @Barcode
     And SortationCode = @SortationCode
End
Else If @Barcode Is Not Null
Begin
   Select Barcode, SortationCode
   From SrtSortationItem With (NoLock)
   Where Barcode = @Barcode
End
Else If @SortationCode Is Not Null
Begin
   Select Barcode, SortationCode
   From SrtSortationItem With (NoLock)
   Where SortationCode = @SortationCode
End
Else
Begin
   Select Barcode, SortationCode
   From SrtSortationItem With (NoLock)
End
```

The optimizer sees four separate queries. Each one compiles a plan suited to its actual filter, with no NULL guard interfering. Each branch caches its own plan and reuses it across calls. Index seeks become possible.

This is the simplest and most maintainable approach when the combinatorics are small. It is also the easiest to read in a year, because each branch states exactly what it does.

The downside is that with N optional parameters, you end up with 2^N branches. At three parameters (eight branches) this is still manageable. At five (thirty-two branches) it is not.

Fix 2: OPTION (RECOMPILE)

Best when the parameter combinations are too many to enumerate but the proc is called rarely enough that the per-call compile cost is acceptable.

```
Select Barcode, SortationCode
From SrtSortationItem With (NoLock)
Where (@Barcode Is Null Or Barcode = @Barcode)
  And (@SortationCode Is Null Or SortationCode = @SortationCode)
Option (Recompile)
```

`Option (Recompile)` tells the engine to skip plan caching and recompile the query on every call. At runtime the parameter values are known, so the engine can substitute them into the predicates and apply parameter embedding. With `@Barcode = 'ABC'` and `@SortationCode = Null`, the predicate effectively becomes `(False Or Barcode = 'ABC') And (True Or ...)`, which the optimizer simplifies to `Barcode = 'ABC'`. That predicate is seekable.

The cost is the recompile itself. A short query like the one above compiles in single-digit milliseconds. For

a proc called millions of times per month, that compile cost adds up. Roughly: at 1 ms per compile and 10 million calls per month, that is 10,000 seconds of CPU spent on compilation. Worth it if the alternative is scanning a large table 10 million times. Not worth it if the proc is called once per second on a small table.

Fix 3: Dynamic SQL with sp_executesql

Best when the parameter count is high (four or more) and IF/ELSE branching would produce an unmanageable number of branches.

```
Declare @Sql nvarchar(max) = N'Select Barcode, SortationCode From SrtSortationItem With (NoLoc

If @Barcode Is Not Null
   Set @Sql = @Sql + N' And Barcode = @Barcode';
If @SortationCode Is Not Null
   Set @Sql = @Sql + N' And SortationCode = @SortationCode';

Exec sp_executesql @Sql,
   N'@Barcode varchar(100), @SortationCode varchar(30)',
   @Barcode = @Barcode, @SortationCode = @SortationCode;
```

The dynamic statement is built per-call to include only the predicates that apply. `sp_executesql` parameterizes the call, so the engine can cache one plan per distinct statement shape. With two optional parameters, four distinct statement shapes are possible, and each gets its own cached plan.

The downside is that dynamic SQL is harder to read, harder to debug, and requires careful attention to SQL injection if any parameter is going to be concatenated rather than parameterized. Always parameterize the values; never concatenate user input into the statement.

Choosing Between the Fixes

Optional parameters	Call frequency	Best fix
1 to 3	any	IF/ELSE branching
4 to 6	any	dynamic SQL with sp_executesql
7+	low (under 100K calls/month)	OPTION (RECOMPILE)
7+	high	dynamic SQL with sp_executesql

The IF/ELSE branch tree is preferred when feasible because it produces the simplest cached plans, the cleanest plan-cache reuse, and the most readable code. Dynamic SQL is the fallback when branch count makes IF/ELSE infeasible. OPTION (RECOMPILE) is the simplest code change but pays a per-call compile cost that scales linearly with call frequency.

Recognizing the Pattern

In existing code, look for any of these signals:

- A WHERE clause with `OR @Param IS NULL` or `OR @Param = ''`.
- Optional parameters with default `= NULL`.
- A proc that runs a scan on every call against a table that has indexes on the predicate columns.
- High avg reads per execution on a proc that returns small result sets.

In Query Store output, the pattern shows up as low plan variability (one plan_id), high logical reads per execution, and CPU time roughly proportional to table scan cost rather than result size.

Index Coverage Still Matters

The catch-all rewrite only delivers if the underlying indexes support the seekable forms. After branching to "barcode only," the engine still needs an index on `Barcode` to seek. After branching to "sortation code only," it needs an index on `SortationCode`. Verify the indexes exist before declaring the refactor a win. If the table is small enough that a scan is cheap (say, under a few hundred rows), the catch-all pattern can be left in place because the cost is negligible. The fix is for tables where scans are expensive.

Related Concepts

- Parameter Sniffing: the related but distinct problem where one cached plan is good for some parameter values and bad for others. The catch-all pattern is upstream of parameter sniffing because it produces one consistently bad plan rather than parameter-dependent variation.
- Non-SARGable Predicates: the broader principle that predicates must be expressed in a form the optimizer can push into an index seek. `Or @Param Is Null` is one specific way to defeat SARGability.